

Comparing TurboQuant+ with traditional quantization methods on multiple LLM

Stanford CS224N {Custom, Default} Project

Matias Soto Carvajal

Faculty of Computer Science
Ruhr-Universität Bochum

Matias.SotoCarvajal@edu.ruhr-uni-bochum.de

Name 2

Faculty of Computer Science
Ruhr-Universität Bochum

name@stanford.edu

Name 3

Faculty of Computer Science
Ruhr-Universität Bochum
name@stanford.edu

Abstract

An abstract should concisely (less than 300 words) motivate the problem, describe your aims, describe your contribution, and highlight your main finding(s).

1 Key Information to include

- Mentor:
- External Collaborators (if you have any):
- Sharing project:

2 Introduction

The introduction explains the problem, why it's difficult, interesting, or important, how and why current methods succeed/fail at the problem, and explains the key ideas of your approach and results. Though an introduction covers similar material as an abstract, the introduction gives more space for motivation, detail, references to existing work, and to capture the reader's interest.

Hoy en día los Large Language Models (LLM) están bastante avanzados. Son capaces de realizar tareas muy complejas de análisis y comprensión de distintas áreas, como también son capaces, gracias a los agentes de IA, de poder codificar, manipular archivos dentro de un sistema operativo, e incluso controlar la misma computadora a través de múltiples herramientas disponibles en el agente de IA. Estas últimas tareas, al ser demasiado complejas, requieren almacenar y procesar una gran cantidad de tokens durante la inferencia y guardar el contexto de la conversación para mantener la coherencia durante la sesión de un agente de IA. Para esto, los modelos utilizan una técnica llamada Key-Value Cache, donde se almacenan las representaciones matemáticas de los tokens anteriores. Gracias a esto, en vez de volver a procesar absolutamente todos los tokens anteriores cada vez que queramos generar un nuevo token, simplemente buscamos en la VRAM del dispositivo las representaciones matemáticas de los tokens anteriores, y usamos este cache para generar los nuevos tokens, ahorrando de manera significativa espacio en la memoria y tiempo. Con esta técnica aplicada, el modelo es capaz de tener ventanas de contexto mucho más largas y mantener la coherencia durante la sesión o conversación.

La gran desventaja de almacenar este cache en la VRAM es que una vez se haya procesado una gran cantidad de tokens durante la sesión del LLM, este cache puede llegar a pesar GB de memoria e incluso superar el uso de memoria de los pesos del modelo mismo, requiriendo que para tareas

complejas y/o de agentes de IA actuales se deba considerar una gran cantidad de VRAM aparte de cargar el modelo mismo.

Para este problema se han buscado multiples soluciones. La mas comun es cuantizar el KV Cache de manera que se pueda reducir el tamaño de la informacion sin que el modelo pierda calidad en su respuestas. La mas simple de estas soluciones es cuantizar los valores Key y Value en niveles de bits mas bajos. Esto tiene una ganancia importante y una perdida de calidad minima al bajar de 16 bits a 8 bits, pero bajo los 8 bits los modelos empiezan a tener una perdida en la calidad de sus respuestas muy notorias, y con niveles de bits muy bajos el modelo se empieza a romper al generar la sintaxis de sus respuestas, provocando que herramientas como el Tool Calling empiecen a fallar de manera muy recurrente. Actualmente el metodo mas utilizado es cuantizar el KV Cache a Q8, puesto que permite reducir bastante el uso de memoria y tener un minimo impacto en la calidad de los modelos.

Frente a este problema es que Google, el 24 de marzo de 2026, lanzo su paper sobre TurboQuant. Un metodo avanzado de optimizacion de memoria.

2.1 TurboQuant

TurboQuant es un metodo que tiene muchas características novedosas que lo hacen teóricamente eficaz con respecto a soluciones mas simples

1. Permite comprimir dinamicamente la memoria durante la ejecucion.
2. Aplica PolarQuant: Rota dinamicamente los vectores de memoria y aplica una transformacion de coordenadas cartesianas a polares. Esta transformacion permite comprimir aun mas la memoria.
3. Utiliza una tecnica de corrección QJL de un solo bit para disminuir la perdida de calidad obtenidas en los pasos anteriores.

Durante los experimentos de esta tecnica los investigadores se dieron cuenta de algo muy importante: **V es libre, K lo es todo.** Solamente es posible cuantizar K hasta un limite razonable (como Q8), pero V soporta compresiones muy agresivas sin perder casi nada de calidad en las respuestas. Esto es revelador ya que, ademas de las transformaciones previas, permite realizar una cuantizacion asincronica en los modelos combinando distintos metodos de compresion tanto en K como en V en busca de un par ideal que maximice el ahorro en memoria y minimize la perdida de la informacion.

De esto tratara principalmente el proyecto actual. Se probara con distintos modelos de LLM distintas combinaciones de cuantizacion de KV Cache y se analizara los resultados para comprobar si efectivamente existe una mejoria notable al momento de usar esta tecnica.

3 Related Work

This section helps the reader understand the research context of your work by providing an overview of existing work in the area.

4 Approach

This section details your approach(es) to the problem. For example, this is where you describe the architecture of your neural network(s), and any other key methods or algorithms.

Para experimentar y descubrir si existe algún beneficio real aplicando las cuantizaciones de TurboQuant es necesario tener una base sólida por donde partir. Los beneficios teóricos de TurboQuant se empiezan a mostrar en la compresión de memoria durante un trabajo de largo contexto.

4.1 Models

Para la ejecucion de los benchmarks se utilizaron los siguientes modelos de peso abierto:

1. Gemma 4 (E4B y E2B) Q8
2. Qwen 3.5 9B Q8

3. Llama 3.1 8B Q8

Todos estos modelos fueron descargados directamente desde HuggingFace. Estos modelos fueron seleccionados ya que son los mas populares dentro de la comunidad de modelos de peso abierto, tienen una buena relacion calidad/tamaño del modelo, y son los mas compatibles con el fork de TurboQuant.

4.2 Framework

Se utilizo el fork Llama-cpp-turboquant. Este fork agrega el codec stack de TurboQuant a Llama.cpp, permitiendo poder cuantizar tanto los pesos como el KV Cache de los modelos. Ademas, al ser un fork directo de Llama.cpp, hereda todo lo bueno de este, como lo es la compatibilidad con distintos backends de inferencia (CUDA, Vulkan, Metal), iniciar servidores de LLM, y la compatibilidad con multiples modelos de distintos proveedores.

4.3 Entorno de trabajo

Todas las pruebas fueron ejecutadas dentro de una instancia Linux rentada desde Vast.ai, con la que se tuvo acceso a una RTX 5090 32GB a unos 100 TFLOPS aproximadamente. Gracias a esto se pudo aumentar la muestra de las pruebas para poder tener resultados mas precisos. Desafortunadamente, el fork de Llama.cpp.turboquant no trae consigo binarios prebuildados de los binarios, por lo que fue necesario buildearlas desde el mismo contenedor para poder utilizar los binarios mas importantes para las pruebas. La conexion se realizo a traves de SSH, en donde se pudo conectar internamente con la instancia, clono el repositorio, y se pudo ejecutar las pruebas.

5 Experiments

This section contains the following.

Las pruebas realizadas en el proyecto fueron las siguientes:

5.1 Data

Describe the dataset(s) you are using (provide references). If it's not already clear, make sure the associated task is clearly described. Being precise about the exact form of the input and output can be very useful for readers attempting to understand your work, especially if you've defined your own task.

El dataset utilizado para ejecutar el benchmark fue LongBench V2 de zai-org. Este dataset tiene 503 problemas de seleccion multiple con una variedad de tematicas, tamaño de contextos y dificultad que lo hacen un desafio muy elevado para los LLM. A favor del tiempo y de los costos de la GPU rentada, se utilizo LongBench de la siguiente manera:

- Se seleccionaron 50 muestras aleatorias. Estas 50 tareas son las mismas para todos los modelos y pares de cuantizacion de KV Cache.
- Estas muestras van desde las tareas cortas, medianas y grandes.
- Se priorizan las tareas cortas y medianas debido a la ventana de contexto definido para los modelos.

5.2 Evaluation method

Describe the evaluation metric(s) you use, plus any other details necessary to understand your evaluation. Some projects will have clear metrics from prior work on given datasets, but we realize that other projects will define their own metrics. If you're defining your own metrics, be clear as to what you're hoping to measure with each evaluation method (whether quantitative or qualitative, automatic or human-defined!), and how it's defined.

Se diseñaron multiples pruebas para los modelos y sus pares de KV Cache para comprobar de manera amplia sobre el rendimiento general de estas cuantizaciones.

Table 1: Idle GPU-memory allocation with traditional and Turbo4 KV caches.

Model	Traditional	Turbo4	Saving	Reduction
Qwen3.5-9B	10,310 MiB	9,662 MiB	648 MiB	6.29%
Llama 3.1 8B	12,900 MiB	10,672 MiB	2,228 MiB	17.27%

5.2.1 LongBench V2

Se diseñó un código Python para correr automáticamente las pruebas en todos los modelos, usando el siguiente orden:

1. Se seleccionan las 50 muestras.
2. Se carga el modelo con sus pesos (Q8_0) con Llama-server.
3. Se define en Llama-server el tipo de cuantización que tendrá los valores de K y V. Los pares seleccionados para las pruebas son:
 - (a) K: F16 V: 16 (Full precision)
 - (b) K: Q8_0 V: Q8_0 (Cuantización más común)
 - (c) K: Q8_0 V: turbo3 (Buen equilibrio entre ahorro y calidad)
 - (d) K: turbo4 V: Q8_0 (Combinación más experimental e inestable)
 - (e) K: turbo4 V: turbo4 (Cuantización más agresiva y delicada de todas)
4. Se ejecutan las 50 tareas en cada combinación de pares KV Cache de cada modelo.
5. Una vez acabado las 50 tareas, se almacenan métricas importantes (Cantidad de respuestas correctas de las 50 tareas, TTFT, Decode and Encode speed, Peak VRAM Usage, etc)
6. Se repite desde el paso dos con los siguientes modelos que quedan durante la ejecución.

5.2.2 Ruler Tests

Fill this

5.2.3 Hermes Agent

We evaluated Qwen3.5-9B and Meta-Llama-3.1-8B-Instruct on an NVIDIA GeForce RTX 5090 using a 65,536-token context window, full GPU offloading, flash attention, and Hermes Agent. Both configurations used the same Q8_0 model weights. The controlled variable within each model pair was the KV-cache format: the traditional configuration used Q8_0 for both the key and value caches, whereas the TurboQuant configuration used Turbo4 for both caches. Consequently, these experiments evaluate Turbo4 KV-cache compression rather than TurboQuant weight quantization.

5.3 Experimental details

Report how you ran your experiments (e.g., model configurations, learning rate, training time, etc.)

5.4 Results

Report the quantitative results that you have found. Use a table or plot to compare results and compare against baselines.

- If you're a default project team, you should **report the accuracy and Pearson correlation scores you obtained on the test leaderboard** in this section. You can also report dev set results if you'd like.
- Comment on your quantitative results. Are they what you expected? Better than you expected? Worse than you expected? Why do you think that is? What does that tell you about your approach?

Table 2: Summary of the autonomous travel-planning results.

Configuration	File generation	Main correctness problem	Result
Qwen traditional	9/9 files	Final cost was €9,290	Failed
Qwen Turbo4	9/9 files	Detailed cost was about €6,947.50	Failed
Llama traditional	1/9 files	Paris-only itinerary and unsupported budget	Failed
Llama Turbo4	Claimed complete	Minimum stated cost was €5,800	Failed

5.4.1 Hermes Agent: Qwen and Llama KV-Cache Comparison

GPU-memory results Turbo4 reduced idle GPU-memory allocation for both models. Qwen3.5-9B saved 648 MiB, corresponding to a 6.29% reduction, while Llama 3.1 8B saved 2,228 MiB, corresponding to a 17.27% reduction. The larger reduction for Llama indicates that the benefit of KV-cache compression depends on the model architecture and cache dimensions. This is the clearest improvement demonstrated by the experiments.

Runtime, utilization, and power For the Qwen travel-planning task, the traditional configuration completed in 2,199.93 seconds, whereas Turbo4 required 2,269.75 seconds. Turbo4 was therefore approximately 69.82 seconds, or 3.17%, slower in this run. Average GPU utilization was similar at 92.50% and 93.33%, respectively. Average power also remained effectively unchanged, decreasing from 563.19 W to 561.48 W. Because the Turbo4 run lasted longer, its estimated energy consumption increased slightly from approximately 0.344 kWh to 0.353 kWh.

For Llama, both configurations repeatedly reached 99–100% GPU utilization and consumed approximately 575 W during active inference. However, the available records did not contain sufficiently consistent execution-time, average-power, or energy measurements for a controlled Llama speed comparison. Therefore, no speed improvement can be claimed for Llama.

The Qwen document-analysis run also does not demonstrate a raw inference speed advantage. The traditional run required approximately 884 seconds, while the recorded Turbo4 run required 2,504.23 seconds and encountered output truncation and four context-compression events. This difference includes agent orchestration, continuation, file operations, and context management, so it must not be interpreted as a direct measurement of token generation speed.

Agent-task quality Neither cache configuration consistently satisfied the autonomous travel task. Both Qwen runs created all nine required files, but both exceeded the €5,000 budget and contained unreliable final verification. The traditional Qwen run calculated a total of €9,290. The Turbo4 Qwen run reported €3,968.50, although its detailed CSV values totaled approximately €6,947.50.

The traditional Llama run performed substantially worse: it created only one of nine required files and produced a 30-day itinerary limited to Paris, despite requiring eight European countries. The Llama Turbo4 response was more complete, but its reported accommodation and transportation costs already totaled €5,800, exceeding the complete task budget before food, attractions, and other expenses were included.

The apparent quality differences cannot be attributed confidently to KV-cache format. Quantization changes memory representation, while agent quality is also affected by sampling variation, generated output length, tool-call behavior, conversation state, and context compression.

6 Analysis

Your report should include *qualitative evaluation*. That is, try to understand your system (e.g., how it works, when it succeeds and when it fails) by inspecting key characteristics or outputs of your model.

6.1 Context behavior

Both model families experienced pressure on the 65,536-token context limit. Hermes used response continuation and context compression when the conversation and tool history became too large. Turbo4 reduces the physical GPU memory occupied by the KV cache, but it does not increase the

logical context limit configured by the server. Consequently, Turbo4 did not prevent output truncation or context compression.

Repeated context compression can also reduce agent reliability because exact requirements, file paths, calculations, and completed steps may be shortened or omitted from the retained summary. This behavior likely contributed to some of the inconsistencies observed in the generated outputs.

6.2 Limitations

The GPU-memory comparison is reasonably controlled because each model pair used the same weights, hardware, context size, and server settings, with the KV-cache format as the intended changed variable. The end-to-end runtime and quality comparisons are less conclusive because most configurations were tested only once, generated-token counts were not matched, tool-call paths differed, context-compression events were inconsistent, and outputs were not verified equally.

Furthermore, `nvidia-smi` measures total GPU allocation rather than the exact amount of KV cache occupied by active tokens. Time to first token, prompt-processing throughput, decode throughput, and exact KV-cache buffer sizes were not measured consistently across all configurations.

7 Conclusion

Summarize the main findings of your project and what you have learned. Highlight your achievements, and note the primary limitations of your work. If you'd like, you can describe avenues for future work.

Turbo4 provided a clear GPU-memory benefit for both evaluated models. It reduced idle allocation by 6.29% for Qwen3.5-9B and 17.27% for Llama 3.1 8B, with Llama saving more than 2.2 GiB. However, the current experiments do not demonstrate a consistent improvement in end-to-end execution time, energy consumption, or autonomous-task quality. The evidence therefore supports Turbo4 primarily as a VRAM-efficiency optimization. Repeated trials with matched token counts, fresh agent sessions, exact KV-cache measurements, time-to-first-token measurements, and decode-throughput measurements are required for a definitive performance comparison.

8 Ethics Statement

What are the ethical challenges and possible societal risks of your project, and what are mitigation strategies?

References

A Appendix (optional)

If you wish, you can include an appendix, which should be part of the main PDF, and does not count towards the 6-8 page limit. Appendices can be useful to supply extra details, examples, figures, results, visualizations, etc. that you couldn't fit into the main paper. However, your grader *does not* have to read your appendix, and you should assume that you will be graded based on the content of the main part of your paper only.